

Wireless Sensors Network

Systèmes communicants pour l'IoT

Project report

7 Janvier 2026

2025/2026 - 5 ISS INSA Toulouse

Amalia Magana IR-SC

Anyia Meetoo IR-SI

Célian Hilal Hamdan IR-SC

Jean-Philippe Loubejac--Combalbert IR-SC

Julie Revelli AE-SE

Marie-Line Da Costa Bento IR-SC

Prof. Daniela Dragomirescu, Gaël Loubet, Lamoussa Sanogo

Summary

Introduction	1
I. Requirements specifications	3
a) Sensor nodes	3
b) Gateway nodes	3
c) Communication Stack	4
i. MAC Layer requirements	4
ii. PHY Layer requirements	4
d) System Level	5
II. Implementation	6
a) MAC layer	6
i. MAC Frame Structure	6
ii. Python implementation	8
b) Physical layer	10
i. From BPSK modulation to D-BPSK modulation	10
ii. Physical Frame Structure	12
iii. GNU Radio Implementation	12
1. Transmission (TX) Flow	12
2. Reception (RX) Flow	14
III. Integrating and testing	15
a) Two USRP Managing	15
b) Compliance with Project Requirements	17
IV. Vectors for improvement	17
a) Adaptive operating modes	17
b) Behaviour-based alerts	17
c) Cooperative Sensing Between Nodes	17
d) Security	17
e) Energy Harvesting from cow movement	18
f) Relay nodes for transhumance scenarios	18
Conclusion	19
References	20
Appendix	21
a) Code repository	21
b) Task management	21

Introduction

This project focuses on designing a wireless sensor network (WSN) for real-time livestock tracking in alpine environments, using software-defined radio (SDR) to develop a custom communication system. The goal is to create a low-power, long-range, and secure solution that transmits GPS coordinates from cattle to a central gateway, addressing the limitations of existing commercial systems. From an educational perspective, this project provides hands-on experience in wireless communication design.

Wireless sensor networks are ideal for monitoring remote or dynamic environments, such as alpine pastures, where traditional infrastructure is lacking. In this context, GPS-enabled sensors attached to cattle transmit their location to a central gateway via a network of antennas. However, designing such a system requires careful consideration of power consumption, coverage, security, and the choice of communication protocols.

The approach taken in this project is based on software-defined radio (SDR), which shifts signal processing from hardware to software. Unlike traditional hardware-defined radio systems, where protocols are fixed and inflexible, SDR allows for reconfigurable modulation schemes, adaptive protocols, and rapid prototyping. By using GNU Radio and USRP hardware, this project implements a custom Physical (PHY) and MAC layer tailored to the specific requirements of livestock tracking.

Several off-the-shelf wireless technologies were considered but ultimately rejected due to their limitations. LoRa, for example, is optimised for long-range, low-power communication but is proprietary and inflexible, making it unsuitable for experimentation. Its low data rate is also unnecessary for transmitting simple GPS coordinates, and its modulation scheme cannot be modified. Ultra-Wideband (UWB) offers high-precision tracking, but it requires a wide bandwidth, which USRPs cannot support, and is too power-intensive for battery-powered sensors. Wi-Fi and Bluetooth, while widely used, are short-range and power-hungry, failing to meet the project's requirements for energy efficiency and coverage. Instead, this project implements a custom solution to achieve the optimal balance of range, power consumption, and flexibility.

The system consists of GPS-enabled sensor nodes attached to cattle, which transmit their location to gateway nodes (USRPs) acting as base stations. These gateways forward the data to a central server for processing and visualisation. The design prioritises low power consumption, enabling operation on small batteries or energy-harvesting sources like solar panels, while ensuring long-range communication (1 to 5 km in mountainous terrain) and security through encrypted transmissions. This project not only addresses the specific use case of livestock tracking but also serves as a template for other low-power sensor networks, such as wildlife monitoring or agricultural sensing.

This report documents the design, implementation, and testing of the system. It begins by outlining the requirements and design choices for the PHY and MAC layers, followed by a detailed description of the implementation process in GNU Radio and Python. The report

concludes with an evaluation of the system's performance and a discussion of potential improvements.

I. Requirements specifications

This section defines the technical and functional requirements for the wireless sensor network, structured around the sensor nodes, gateway, and communication protocols. The goal is to ensure the system meets the demands of low-power operation, long-range communication, and secure data transmission in alpine environments.

The system is divided into three main components: sensor nodes (attached to cattle), gateways (USRPs acting as base stations), and the communication stack (PHY and MAC layers). Each component has specific requirements to ensure the overall system operates efficiently and reliably.

a) Sensor nodes

Sensor nodes are responsible for collecting and transmitting GPS data while operating under strict power constraints. Key requirements include:

- **GPS Tracking:** Each node must transmit latitude, longitude, altitude, and a unique cow ID at regular intervals of 10 minutes because we don't need to have that many values to be able to locate the cows.
- **Power Supply:** Nodes must operate for extended periods using small batteries or energy-harvesting methods, such as:
 - **Solar panels:** To recharge batteries during daylight.
 - **Piezoelectric generators:** To harvest energy from cattle movement.
 - **Mechanical energy harvesters:** To capture energy from environmental vibrations or motion.
- **Power Consumption:** Nodes must consume less than $\sim 100\text{ mW}$ during active transmission and GPS fix operations and enter sleep mode between transmissions to conserve energy, since typical GPS tracking modules draw on the order of tens of milliamps when acquiring a fix but can average $\sim 10\text{--}30\text{ mA}$ ($\sim 30\text{--}100\text{ mW}$) [1] over a 10-minute cycle with deep sleep between fixes.
- **Compact and Lightweight Design:** Nodes must be small and lightweight to avoid discomfort for the animals.
- **Security:** Transmissions must be encrypted to prevent eavesdropping or spoofing over the cow localisation.

b) Gateway nodes

Gateway nodes (USRPs) act as intermediaries between the sensor nodes and the central server. Their requirements include:

- **Long-Range Communication:**
- Gateways must reliably receive transmissions from sensor nodes located up to 5 km away under line-of-sight conditions, while also accounting for signal degradation caused by mountainous terrain. This range is sufficient to cover large alpine pastures,

which typically span at least 10 hectares [2], requiring wide-area wireless coverage from a limited number of gateways.

- **Data Aggregation:** Gateways must aggregate data from multiple sensor nodes and forward it to the central server for processing.
- **Reliability:** Gateways must ensure robust communication with an acceptable packet loss rate of up to 30%, while still guaranteeing that each sensor node successfully delivers at least one location update every 20 minutes to maintain continuous livestock tracking.
- **Power Supply:** Gateways should be powered by a stable source to avoid downtime.

c) Communication Stack

The communication stack is divided into the MAC layer and the PHY layer, each with specific requirements to ensure efficient and reliable data transmission.

i. MAC Layer requirements

The MAC layer is responsible for framing, addressing, error detection, and channel access. Key requirements include:

- **Frame Structure:** Data must be organised into fixed-length frames for efficient transmission.
- **Error Handling:** Frame integrity must be verified using a CRC-32 checksum. Corrupted frames are discarded upon detection, as no retransmission mechanism is implemented at the sensor node level to reduce energy consumption.
- **Collision Handling:** To mitigate packet collisions in the absence of coordination, each data transmission is repeated three times, increasing the probability of successful reception without requiring acknowledgements.
- **Addressing:** Each transmission must include a 32-bit cow ID to identify the source.
- **Security:** Data must be encrypted using AES-128, for example, to prevent unauthorised access.
- **Channel Access:** A pure ALOHA access scheme is used, as transmissions occur only once every 10 minutes and the network is limited to approximately 100 sensor nodes, resulting in a low channel load.

ii. PHY Layer requirements

The PHY layer handles modulation, synchronisation, and error correction. Key requirements include:

- **Modulation Scheme:** DBPSK (Differential Binary Phase-Shift Keying) is selected to favour long-range and robust communication over high data rates.
- **Data Rate:** The target data rate is approximately 1 bit/s, which is sufficient to transmit 48 bytes every 10 minutes; $((48 \times 8) / 10 \times 60) = 0.64 \text{ bits/s}$.
- **Synchronization:** The receiver must detect the start of a transmission using a 128-bit preamble and a 32-bit Start of Frame Delimiter (SFD).
- **Bandwidth:** The system must operate within a 500 kHz bandwidth to avoid interference. This allows the geolocation sensors to sample at 1 MHz (per Shannon's

theorem), ensuring high-resolution data capture while avoiding interference with other wireless devices in the alpine environment.

- **Carrier Frequency:** Communication will use the 433 MHz ISM band, chosen for its favourable propagation characteristics and robustness in mountainous environments. The system complies with regulatory constraints, including a maximum transmit power of 10 dBm and a maximum duty cycle of 10%.[3]
- **Error Correction:** No forward error correction (FEC) is implemented, as reliability is instead achieved through repeated transmissions and CRC-based error detection.
- **Sub-channels:** Sub-channelization is not required due to the low node density and minimal spectrum usage typical of remote mountainous areas.

d) System Level

The system must meet the following overarching requirements:

- **Range:** Achieve a communication range of **1–5 km** in mountainous terrain.
- **Scalability:** Support **dozens of sensor nodes** without significant degradation in performance.

II. Implementation

This section details the implementation of the physical (PHY) layer in GNU Radio and the medium access control (MAC) layer as a Python block, focusing on frame structure, error handling, and synchronisation.

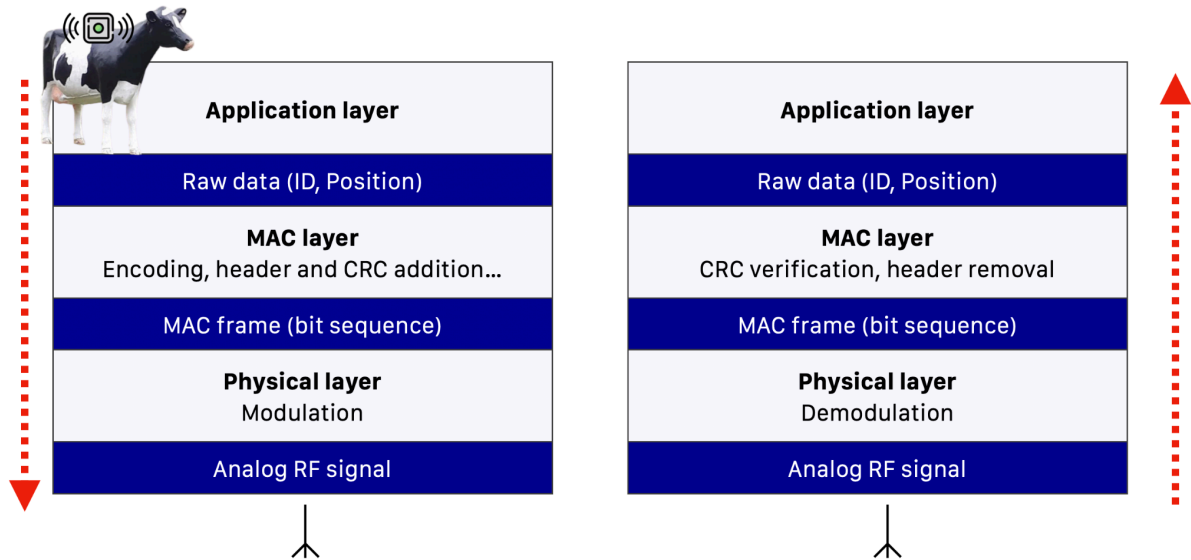


Figure 1. Data flows through the different layers

a) MAC layer

The MAC layer is responsible for organising data into frames, managing access to the shared medium, and ensuring reliable transmission. For this project, we designed a custom MAC frame tailored to the needs of GPS-based livestock tracking, balancing simplicity, efficiency, and security.

i. MAC Frame Structure

The MAC frame consists of (22 bytes):

1. **Header** (1 bytes):
 - **Address** (8 bits): an address for the gateway, generated randomly.
2. **Payload** (16 bytes = 128 bits):
 - **Cow ID** (32 bits): Unique identifier for each animal.
 - **Latitude** (32 bits): GPS coordinate in fixed-point format.
 - **Longitude** (32 bits): Same format as latitude.
 - **Altitude** (32 bits): Meters above sea level.
3. **Footer** (5 bytes = 32 + 8 bits):
 - **Padding** (8 bits) : 0x00.
 - **CRC-32**: Cyclic redundancy check or CRC is used to detect transmission or transfer errors. It is evaluated before transmission or transfer and written on a

few bits at the end of the frame. After transmission, it is recalculated and compared to the end-of-frame figure to ensure that the data is likely identical.

Here is an example of a MAC frame and its interpretation: **2b00 000a 9502 e43d 5900 20e8 f500 0000 8200 c904 1ae6**.

Section	Field Name	Size	Hex Value	Interpreted Value
Header	Gateway Address	1 Byte	2b	43 (Decimal ID)
Payload	Cow ID	4 Bytes	00 00 0a 95	2709 (Unique ID)
	Latitude	4 Bytes	02 e4 3d 59	48.512345
	Longitude	4 Bytes	00 20 e8 f5	2.156789
	Altitude	4 Bytes	00 00 00 82	130 meters
Footer	Padding	1 Byte	00	0x00
	CRC-32	4 Bytes	c9 04 1a e6	3,372,489,446 (Checksum)

Figure 2. MAC Frame interpretation

This cow is located at Chemin du Gros Buisson, in Souzy-la-Briche, France.

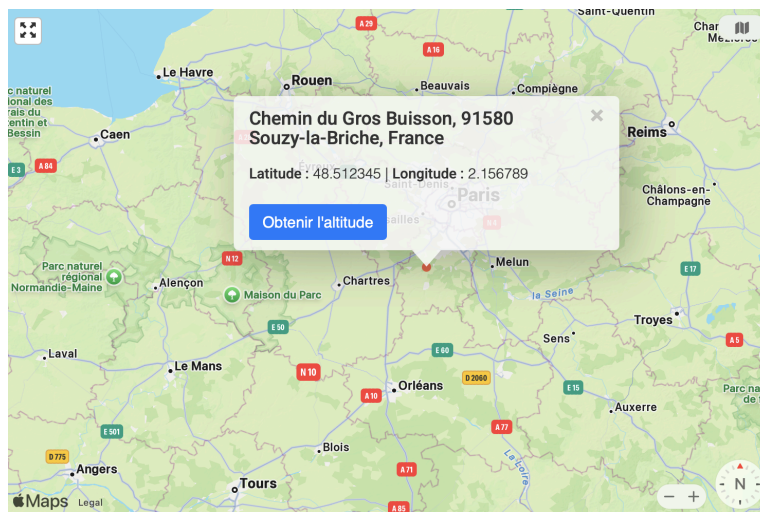


Figure 3. Cow location.



Figure 4. Indeed, the cow was there!

ii. Python implementation

We choose python to implement the MAC layer, to simplify the interface between the two layers. In GNU Radio Companion, it's possible to add blocks who execute Python code directly. To add the MAC layer, we added two Python blocks to the GNU Radio file:

- The first block, Random MAC Frame Generator, generates MAC frames with random parameters. It returns the frame as a byte. Here is the code; the `\_\_init\_\_` function is executed only once at the start of the block, and then the `work` function is called repeatedly. The code is also available on GitHub.

```
import numpy as np
import struct
import zlib
import random
from gnuradio import gr
import pmt

# taille d'une frame (1 byte address + 4*4 bytes data + 1 byte padding + 4 bytes CRC)
FRAME_LEN = 1 + 4 * 4 + 1 + 4

class blk(gr.sync_block):
    def __init__(self, address=0x2B, cow_id=2709, min_lat=48000000, max_lat=49000000, min_lon=15000000, max_lon=25000000, min_alt=100, max_alt=200):
        """Arguments de la fonction qui apparaissent comme paramètres dans GRC"""
        gr.sync_block.__init__(
            self,
            name="Random MAC Frame Generator",
            in_sig=None,
            out_sig=[np.byte]
        )

        # initialisation des paramètres
        self.address = int(address)
        self.cow_id = int(cow_id)
        self.min_lat = int(min_lat)
        self.max_lat = int(max_lat)
        self.min_lon = int(min_lon)
        self.max_lon = int(max_lon)
        self.min_alt = int(min_alt)
        self.max_alt = int(max_alt)

        self.frame_len = FRAME_LEN
```

Figure 5 . __init__ function

```

def generate_random_frame(self):
    # genere des valeurs aléatoires
    latitude = random.randint(self.min_lat, self.max_lat)
    longitude = random.randint(self.min_lon, self.max_lon)
    altitude = random.randint(self.min_alt, self.max_alt)

    # payload (header + données)
    # Format: >8IIII (Big-endian: 1 byte + 4 * 4 bytes)
    payload_bytes = struct.pack(">8IIII", self.address, self.cow_id, latitude, longitude, altitude)

    # ajoute 1 octet de bourrage
    payload_bytes += b"\x00"

    # calcule le CRC (sur tout sauf le CRC lui-même)
    # & 0xFFFFFFFF est pour assurer une valeur non signée de 32 bits
    crc = zlib.crc32(payload_bytes) & 0xFFFFFFFF

    # frame complète = payload + bourrage + CRC
    # Format pour le CRC: >I (Big-endian: 4 bytes)
    frame = payload_bytes + struct.pack(">I", crc)

    return frame

def work(self, input_items, output_items):
    out = output_items[0]
    frame_len = FRAME_LEN
    if len(out) < frame_len:
        return 0

    frame_bytes = self.generate_random_frame()
    frame_np = np.frombuffer(frame_bytes, dtype=np.byte)

    out[frame_len] = frame_np

    self.add_item_tag(0,
                      self.nitems_written(0),
                      pmt.intern("packet_len"),
                      pmt.from_long(frame_len))

    return frame_len

```

Figure 6. work function

Note that on GNU Radio Companion, it is possible to enter the cow ID as well as the minimum and maximum of all parameters (latitude, longitude and altitude) to generate random frames.

Properties: Random MAC Frame Generator	
General	Advanced Documentation
Code	Open in Editor Choose Editor
Address	0x2B
Cow_Id	2709
Min_Lat	48000000
Max_Lat	49000000
Min_Lon	1500000
Max_Lon	2500000
Min_Alt	100
Max_Alt	300
Valider Annuler Appliquer	

Figure 7. Entering parameters in GNU C

- The second block, MAC Decoder & Display, decodes and displays the received frame. The code checks the CRC and displays the interpretation of the received frame. Here is the code; it is, of course, available on GitHub.

```

import numpy as np
from gnuradio import gr
import struct
import zlib

class bik(gr.sync_block):
    def __init__(self):
        gr.sync_block.__init__(
            self,
            name='MAC Decoder & Display',
            in_sig=[np.uint8],
            out_sig=None
        )
        self.frame_size = 22
        self.buffer = bytearray()

        print("\n" + "="*60)
        print(f"{'Frame':<8} | {'Addr':<6} | {'CowID':<8} | {'Lat':<10} | {'Long':<10} | {'Alt':<10} | {'CRC':<6}")
        print("-" * 60)

    def work(self, input_items, output_items):
        in_data = input_items[0]

        self.buffer.extend(in_data.tobytes())

        # tant qu'on a assez de données pour au moins une trame
        while len(self.buffer) >= self.frame_size:
            trame = self.buffer[:self.frame_size]
            self.buffer = self.buffer[self.frame_size:] # On vide ce qu'on a lu

            try:
                res = struct.unpack(">BIIIBI", trame)
                address, cow_id, lat, lon, alt, padding, crc_rx = res

                # vérification CRC
                crc_calc = zlib.crc32(trame[:-4]) & 0xFFFFFFFF
                crc_ok = "OK" if crc_rx == crc_calc else "BAD"

                print(f"RECUE | {hex(address):<6} | {cow_id:<8} | {lat:<10} | {lon:<10} | {alt:<10} | {crc_ok:<6}")

            except Exception as e:
                print(f"Erreur décodage: {e}")

        return len(in_data)

```

Figure 8. Program to decode and display the received frames

b) Physical layer

The physical layer is responsible for transmitting MAC frames over the air, handling modulation, synchronisation, and error correction. We implemented the PHY layer in GNU Radio, using a DBPSK modulation scheme. In the following, you will find GNU Radio blocks in underlined green, variables in **green** and GNU Radio functions in *italic green*.

i. From BPSK modulation to D-BPSK modulation

BPSK modulation has two distinct modulation states, represented on the constellation diagram by two constellation points. Each of these two constellation points corresponds to the transmission of a single bit (0 or 1).

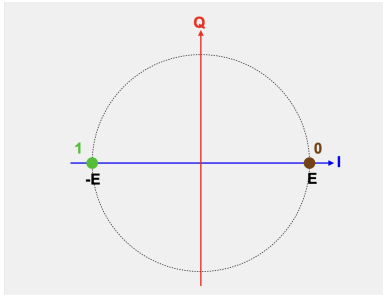


Figure 9. BPSK Modulation

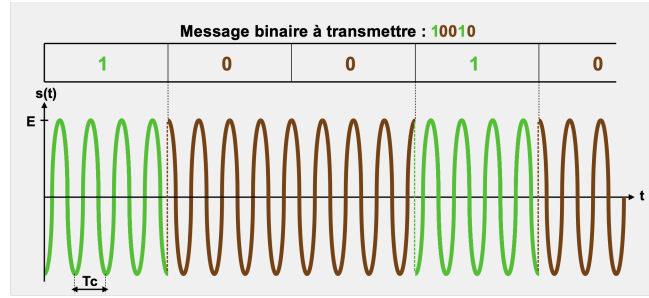


Figure 10. Modulated signal $s(t)$ with BPSK

DBPSK (Differential Binary Phase Shift Keying) modulation has a constellation diagram identical to that associated with BPSK modulation, but the encoding of the transmitted information is different. If a 0 is to be transmitted, the previous phase is maintained. If a 1 is to be transmitted, a phase rotation of π is applied.

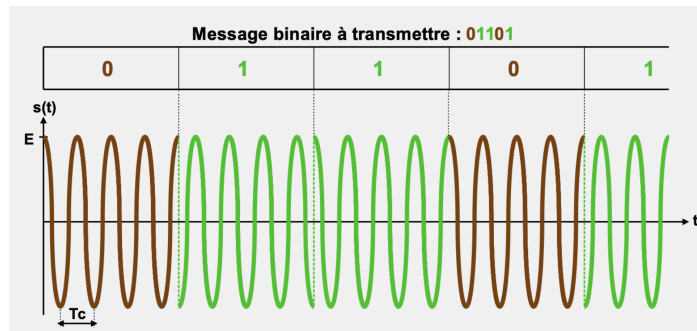


Figure 11. Modulated signal $s(t)$ with DBPSK

DBPSK modulation, therefore, encodes information not in the absolute phase of the signal but in the phase difference between two successive symbols. This allows the receiver to dispense with an absolute phase reference, reducing synchronisation complexity. We chose D-BPSK (Differential Binary Phase-Shift Keying) modulation for its robustness to phase offsets and noisy environments, which is the case near coffee machines and in rooms due to disruptive foot traffic.

The addition of a preamble in a signal processing chain mainly serves to ensure reliable transmission and reception. It uses a binary sequence known to all. It is inserted at the beginning of each packet to enable synchronisation at the receiver. It therefore acts as a reference allowing the receiver to correctly decode the DBPSK-modulated data. In a noisy environment or one subject to frequency offsets, the preamble allows the receiver to correct these offsets before decoding the data.

ii. Physical Frame Structure

To ensure synchronization and reliable transmission, we prepend a physical header to the MAC frame. The total physical frame size is 42 bytes:

1. **Preamble** (128 bits): A fixed bit pattern used for receiver synchronization and automatic gain control ([AGC](#)). The preamble helps the receiver detect the presence of a signal and lock onto its frequency and phase. The preamble is generated using the following key: `111000010101101010101010101110100010010011`. This pattern is generated by the `digital.header_format_default(key, 0)` function in GNU Radio. This function creates a standardized header that includes the preamble and SFD, ensuring the receiver can synchronize and detect the start of the frame.
2. **Start of Frame Delimiter** (SFD, 32 bits in the preamble): A unique bit sequence marking the end of the preamble and the start of the MAC frame. The SFD ensures the receiver correctly identifies the beginning of the data payload. The SFD is part of the header generated by the `digital.header_format_default` function.
3. **MAC Frame** (160 bits): The MAC frame (header + payload + CRC) as described in MAC Frame Structure Section.

iii. GNU Radio Implementation

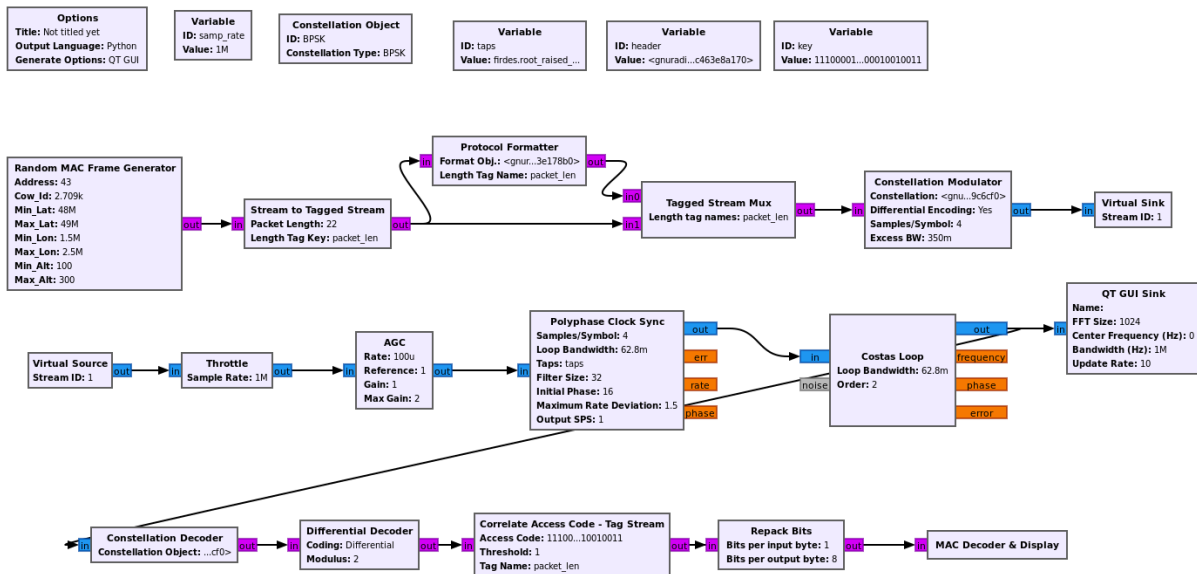


Figure 12. Gnu radio file for initial physical layer

1. Transmission (TX) Flow

The main idea of this part is to modulate the signal to make it suitable for transmission over the channel. The following frame is taken as an example:

2b00 000a 9502 e43d 5900 20e8 f500 0000 8200 c904 1ae6

Random MAC Frame (Python block): Generate random MAC frame and return as binary.

At the output, we will have the following 176-bit bitstream:

```
2b = 00101011
00 = 00000000
00 = 00000000
0a = 00001010
...
1a = 00011010
e6 = 11100110
```

Stream to Tagged Stream: Tags the stream with the packet length for further processing. Packet length is set to 22 bytes.

Protocol Formatter: Adds the physical header (preamble including SFD) to the MAC frame. Uses the preamble key.

Here we use *here header_format_default(key,0)* with "1110000101011010101010101110100010010011" as a key. The header contains: the Access Code + length (16 bits) + repeated length (16 bits).

The length is 22 bytes, so 00000000 00010110 in binary.

Therefore, the header content (74 bits) is:

1110000101011010101010101110100010010011 (Key) 0000000000010110
(length) 0000000000010110 (repeated length)

Tagged Stream Mux: Multiplexes the tagged stream for modulation.

Here, the two parts are stuck together. This block defines the complete physical frame.

The physical frame therefore looks like this: [11100001...0110] [00101011 00000000 00000000 00001010 ...] (The first byte 2b has become 00101011).

Constellation Modulator (DBPSK): The combined frame is modulated using DBPSK. DBPSK is chosen for its robustness in noisy environments and simplicity in implementation.

We are now entering the analog domain (complex numbers). Each bit becomes a symbol. Since we have 4 samples/symbol, the physical frame's content will be like that :

If the bit is 0 → Phase 0 (for example $1+0j$)

If the bit is 1 → Phase π (for example $-1+0j$)

Each bit is repeated 4 times. [(1,0), (1,0), (1,0), (1,0), (-1,0), (-1,0), (-1,0), (-1,0), ...]

Throttle: Limits the data rate to 1 bit/s (1 sample per second), as required by the project specifications.

Virtual Sink(TX_RX_0): Simulates the transmission of the modulated signal.

2. Reception (RX) Flow

The main idea of this part is to demodulate the signal to decode it:

1. Virtual Source(TX_RX_1): Simulates the reception of the modulated signal.
2. AGC (Automatic Gain Control): Adjusts the signal amplitude to optimise dynamic range. We took a reference of 1, to set the target output signal level to unity (1), a gain of 1 to be neutral (no amplification or attenuation) and a maximum amplification to 2 (6 dB). This prevents excessive noise amplification or distortion if the input signal is very weak.
3. Polyphase Clock Sync: Synchronises the transmitter and receiver clocks to ensure accurate symbol timing. We took 4 Samples/Symbol to improve timing resolution and reduce jitter, a loop bandwidth of 100m, which is a narrow bandwidth that filters out noise, often chosen as a fraction of the symbol rate (1M), and 32 Taps to ensure accurate symbol timing recovery by using a filter length of 32.

It reduces the flow (decimation) to deliver only one sample per symbol, thus eliminating the repetition of samples done during modulation.

4. Costas Loop: Used for carrier phase recovery, which is critical for coherent demodulation. We took a loop bandwidth of 62.8m to be coherent with the polyphase block and an order of 2 to loop tracks both phase and frequency offsets, which is essential for coherent demodulation.
5. Constellation Decoder(DBPSK): Demodulates the received signal back to bits.
6. Differential Decoder: Decodes the differentially encoded bits.
7. Correlate Access Code: Detects the preamble to identify the start of the frame. It has the preamble as parameter Access Code: 1110000101011010101010101011101

The frame's content (in bits) is as follows:

0010101100000000000000000000000010101001010100000010...

8. Repack Bits: This block takes the previous bits and groups them into groups of 8. It converts the demodulated bits into bytes.

The frame's content (in bytes) is as follows: 2b 00 00 0a 95 02 e4 3d 59 00 20 e8 f5 00 00 00 82 00 c9 04 1a e6

9. MAC Decoder & Display (Python block): Decodes the frame, calculates the CRC, and displays the data sent.

III. Integrating and testing

a) Two USRP Managing

We divided the GNU Radio file into two files to transmit and receive data:

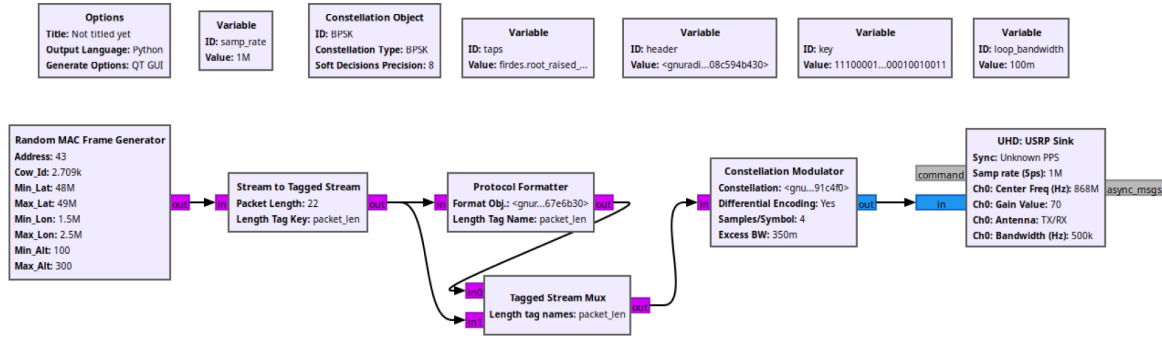


Figure 13. Transmitting random MAC frames with GNU Radio

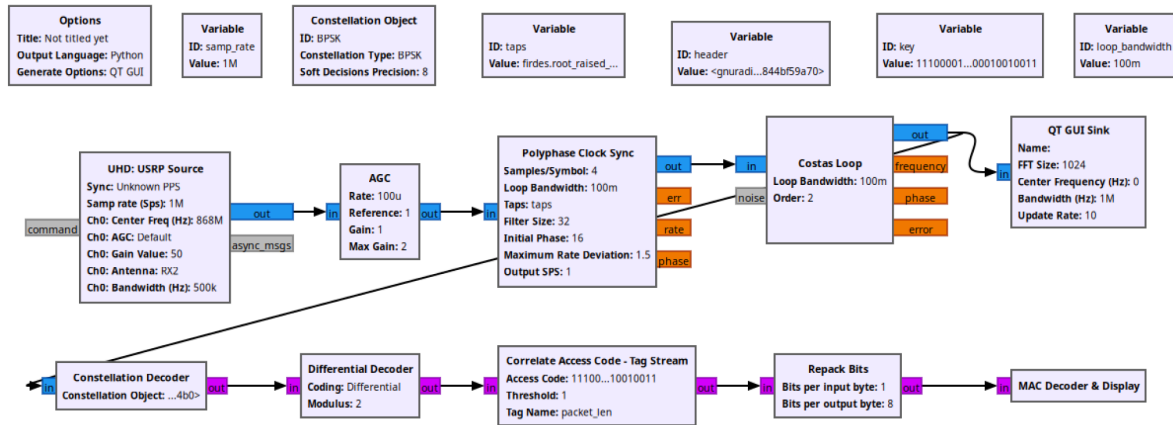


Figure 14. Receiving random MAC frames in GNU Radio



Figure 15. A photo of the transmission between 2 USRPs

In this implementation, we tried to send random frames, and it seems to work. You can find the entire file here:

https://github.com/millilitre/Wireless_Sensor_Network/blob/main/phy/tramerecue_decoded.txt.

Trame	Addr	CowID	Lat	Long	Alt	CRC
RECUE	0x2b	2709	48177739	1681592	116	OK
RECUE	0x2b	2709	48396867	2129676	175	OK
RECUE	0x2b	2709	48128086	2017372	210	OK
RECUE	0x2b	2709	48735843	1893599	106	OK
RECUE	0x2b	2709	48569600	2206581	242	OK
RECUE	0x2b	2709	48086462	2351720	261	OK
RECUE	0x2b	2709	48030954	1560456	287	OK
RECUE	0x2b	2709	48625609	2173245	131	OK
RECUE	0x2b	2709	48479808	1789515	285	OK
RECUE	0x2b	2709	48005230	2269023	223	OK
RECUE	0x2b	2709	48449455	1598613	115	OK
RECUE	0x2b	2709	48736945	1872799	160	OK
RECUE	0x2b	2709	48526278	2224105	183	OK
RECUE	0x2b	2709	48920511	2101562	222	OK
RECUE	0x2b	2709	48946579	2333152	289	OK
RECUE	0x2b	2709	48749112	1812560	257	OK
RECUE	0x2b	2709	48669552	1552599	227	OK
RECUE	0x2b	2709	48625330	1633321	194	OK
RECUE	0x2b	2709	48005360	2488876	139	OK
RECUE	0x2b	2709	48019991	2059488	226	OK
RECUE	0x2b	2709	48780899	1869719	103	OK
RECUE	0x2b	2709	48435217	2002944	248	OK
RECUE	0x2b	2709	48276024	1854829	167	OK
RECUE	0x2b	2709	48601713	1544060	111	OK

Figure 16. An example of received random data, related to the cow 2709.

b) Compliance with Project Requirements

- **Robustness:** DBPSK is resilient to phase ambiguities and performs well in multipath environments, which is crucial for mountainous terrain.
- **Data Rate:** The target data rate of 1 bit/s is sufficient for transmitting 48 bytes every 10 minutes, as required by the project specifications.
- **Long-Range Communication:** DBPSK and the 433 MHz band ensure reliable communication up to 5 km in line-of-sight conditions.
- **Low Power Consumption:** The simple modulation scheme and low data rate minimise power usage, aligning with the sensor node's power constraints.
- **Scalability:** The system supports dozens of nodes with minimal spectrum usage, as required for alpine pastures.

IV. Vectors for improvement

Several improvements and future extensions could enhance the performance, robustness, and functionality of the proposed system.

a) Adaptive operating modes

A secondary operating mode could be introduced and activated when a cow is detected as being unusually far from the rest of the herd or when a sensor node stops responding. In this mode, the node would transmit data more frequently or include additional information to improve localisation and facilitate recovery.

b) Behaviour-based alerts

If a cow remains stationary for an extended period, the system could flag this behaviour as abnormal, potentially indicating injury, illness, or device malfunction, and generate an alert to the central server.

c) Cooperative Sensing Between Nodes

Sensor nodes could exchange limited information locally and relay it to the gateway. Nodes located at the edge of the network would transmit only when an anomaly is detected, for example, by estimating their distance to the nearest neighbouring cow, thereby reducing unnecessary transmissions.

d) Security

While AES-based encryption could significantly improve security, it was not fully implemented in this prototype due to time constraints. Its inclusion would increase computational load and energy consumption, requiring careful optimisation or hardware acceleration.

e) Energy Harvesting from cow movement

Mechanical energy harvesting using the oscillatory motion of the cow's bell could be explored. By exploiting resonance frequencies, this approach could both recharge the battery and help identify individual bells, indirectly contributing to cow identification.

f) Relay nodes for transhumance scenarios

Additional relay or temporary gateway nodes could be deployed in alpine pastures during transhumance periods, where animals move to higher altitudes and direct communication with fixed gateways becomes unreliable.

Conclusion

This project demonstrated the feasibility of designing a custom wireless sensor network for real-time cattle tracking in alpine environments using Software-Defined Radio (SDR). By developing a communication stack featuring DBPSK modulation, a 500 kHz bandwidth, and a lightweight MAC layer with CRC-32 error detection, the system effectively balances low power consumption, long-range communication, and reliability in challenging mountainous terrain.

The implementation, validated using USRP hardware and GNU Radio, highlighted the advantages of a reconfigurable and energy-efficient approach. Through this project, the team gained practical experience in wireless communication protocols, hardware-software integration, and collaborative development, reinforcing the importance of interdisciplinary collaboration in addressing real-world IoT challenges.

While the current prototype meets core requirements, opportunities for enhancement remain. Future work could explore AES encryption for improved security, adaptive transmission modes for anomaly detection, and relay nodes to extend coverage during transhumance. Additionally, integrating cooperative sensing or advanced energy-harvesting techniques could further optimise performance.

References

- [1] https://www.skylabmodule.com/product/small-size-gps-timing-module-skg12at/#label_2
- [2] sg-proxy02.maafr.ate.info
- [3] https://www.arcep.fr/uploads/tx_gsavis/21-1589.pdf

Appendix

a) Code repository

All of the project code can be found here:

https://github.com/millilitre/Wireless_Sensor_Network.git.

b) Task management

The project tasks were distributed among six team members to ensure clear responsibilities and efficient parallel development across the physical and MAC layers:

- **Physical Transmission:** Hilal and Jean-Philippe were responsible for the physical transmission chain, including signal generation, modulation implementation, and transmission parameter validation at the PHY layer.
- **Physical Reception:** Marie-Line and Julie were in charge of the physical reception chain, including signal detection, synchronisation, demodulation, and performance evaluation under realistic channel conditions.
- **MAC Layer Design:** Anya and Amalia were responsible for the MAC layer, including frame structure definition, channel access strategy, collision mitigation mechanisms, and addressing.

However, we all contributed to all parties to ensure that we understood the project and helped each other. This makes sense since we didn't finish every part of the project at the same time.